



University of Management & Technology

Department of Computer Science (SST/CS)

PF Lab Project

Documentation of Simple Banking System

Teacher: Bilal Hussain

Course Title: Programming Fundamentals Lab

Course Code: CC1021L

Deadline: 31st Jan 2024

Members:

Muhammad Sadeed Ahmad F2023266183

Afra Fatima F2023266176

Rania Waseem F2023266153

Adeena Khan F2023266164

This C++ code represents a simple banking system that allows users to perform various operations, such as viewing account details, opening new accounts, closing accounts, checking balances, depositing amounts, and withdrawing funds. Below is a detailed explanation of the code:

Header Files

```
#include <iostream>
#include <cstdlib>
#include <ctime>
```

These lines include necessary header files for input/output, random number generation, and time functions for seeding random number with time.

Namespace

```
using namespace std;
```

This line avoids the need to prefix standard C++ libraries with **std::**.

Constants

```
const int max_users = 1000;
```

Defines a constant **max_users** representing the maximum number of users/accounts the system can handle.

Main Function

```
int main()
```

The main function begins here.

Variable Declarations

```
string enter_name, name[max_users] = { "Sadeed" };
int enter_cnic, cnic[max_users] = { 123 };
int enter_accnum, accnum, account_number[max_users] = { 111 };
int access, enter_pin, pin[max_users] = { 9999 };
long int withdraw, deposit, savings[max_users] = { 1200 }, defaults[max_users] = {
500 }, current[max_users] = { 1500 };
int option;
int userindex = -1;
```

- **name**, **cnic**, **account_number**, **pin**, **savings**, **defaults**, and **current** are arrays to store user information, such as name, CNIC, account number, PIN, and account balances.
- **option** stores the user's menu choice.
- **userindex** is used to track the index of the user account in various arrays.

Main Menu Using Switch Statement

start:

```
    cout<<"1 Account Detail \n2 Open an Account \n3 Close an Account \n4 Show
Balance \n5 Deposit Amount \n6 Withdraw Amount \n7 Exit \nSelect Option: ";
    cin>>option;
    cout<<endl;
```

```
switch(option)
{
    // Cases for different menu options...
}
```

This part presents a menu to the user, which has options like 1 Account Detail, 2 Open an Account, 3 Close an Account, 4 Show Balance, 5 Deposit Amount, 6 Withdraw Amount, 7 Exit. The user can select the option by entering the number correlated to it.

Case 1: Account Detail

case 1:

start1:

```
    userindex = -1;
    cout << "Enter your CNIC Number: ";
    cin >> enter_cnic;

    for (int i = 0; i < max_users; i++) {
        if (enter_cnic == cnic[i]) {
            userindex = i;
            break;
        }
    }

    if (userindex != -1) {
        cout << "Account found!" << endl << "Details:" << endl;
        cout << "Account Owner: " << name[userindex] << endl;
        cout << "Account Number: " << account_number[userindex] << endl;
        cout << "Account Pin: " << pin[userindex] << endl << endl;
        goto start;
    } else {
        cout << "No Record Found!" << endl << endl;

        nav1:
            cout << "1 Back \n2 Main Menu \n3 Exit \nSelect Option: ";
            cin >> option;

            switch (option) {
                case 1:
                    cout << endl;
                    goto start1;
                    break;
                case 2:
                    cout << endl;
                    goto start;
                    break;
                case 3:
                    cout << "Exiting System!";
                    return 0;
                    break;
                default:
                    cout << "Invalid Selection!" << endl;
                    goto nav1;
                    break;
            }
    }
}
```

```
break;
```

Explanation:

1. **Initialization:**
 - **userindex** is initialized to -1 to indicate that the user hasn't been found initially.
2. **User Input:**
 - The program prompts the user to enter their CNIC (National Identity Card) number.
3. **Search for User:**
 - It then iterates through the array of CNIC numbers (**cnic**) to find a match.
 - If a match is found, the index (**userindex**) of the user is updated, and the loop breaks.
4. **Display Account Details:**
 - If the user is found (**userindex != -1**), the program displays the details of the account, including the account owner's name, account number, and PIN.
 - After displaying the details, it goes back to the main menu (**goto start;**).
5. **No Record Found:**
 - If no user is found, it informs the user that no record is found.
6. **Navigation Options:**
 - It provides options to the user:
 - **1** to go back and try entering the CNIC again (**goto start1;**),
 - **2** to go back to the main menu (**goto start;**),
 - **3** to exit the program.
7. **Handling Navigation:**
 - Depending on the user's choice, the program either goes back to entering the CNIC (**start1**), back to the main menu (**start**), or exits the program.

This case allows the user to check their account details by entering their CNIC number. If the CNIC is found, it displays the account information; otherwise, it provides options for navigation. The use of **goto** here allows for a simple way to manage user navigation within the program.

Case 2: Open an Account

case 2:

```
start2:
    userindex = -1;
    nam:
        getline(cin, enter_name);
        if (enter_name.empty()) {
            cout << "Enter your name: ";
            goto nam;
        }
        cout << "Enter your CNIC number (3 digits): ";
        cin >> enter_cnic;

    for (int i = 0; i < max_users; i++) {
```

```

        if (enter_cnic == cnic[i]) {
            userindex = i;
            break;
        }
    }

    if (userindex != -1) {
        cout << "An account is already registered to this CNIC!";

        nav2:
            cout << "\n\n1 Back \n2 Main Menu \n3 Exit \nSelect Option: ";
            cin >> option;

        switch (option) {
            case 1:
                cout << endl;
                goto start2;
                break;
            case 2:
                cout << endl;
                goto start;
                break;
            case 3:
                cout << "Exiting System!";
                return 0;
                break;
            default:
                cout << "Invalid Selection!" << endl;
                goto nav2;
                break;
        }
    } else {
        for (int i = 0; i < max_users; i++) {
            if (cnic[i] == 0) {
                name[i] = enter_name;
                cnic[i] = enter_cnic;

                again:
                    srand(time(0));
                    accnum = rand() % 1000 + 100;
                    for (int j = 0; j < max_users; j++) {
                        if (accnum == account_number[j])
                            goto again;
                    }

                    account_number[i] = accnum;
            }
        }
    }
}

```

```

        create:
            cout << "Create a 4 digit pin: ";
            cin >> enter_pin;

            if (enter_pin > 999 && enter_pin < 10000) {
                cout << "Pin created!";
                pin[i] = enter_pin;
                cout << endl << endl;
                goto start;
            } else {
                cout << "Please enter a 4 digit pin!" << endl << endl;
                goto create;
            }
            break;
        }
    }
    break;

```

Explanation:

1. **Initialization:**
 - **userindex** is initialized to -1 to indicate that the user hasn't been found initially.
2. **User Input:**
 - The program prompts the user to enter their name and a 3-digit CNIC number.
3. **Search for Existing Account:**
 - It then checks if there is already an account registered with the provided CNIC.
4. **Account Already Exists:**
 - If an account is found (**userindex != -1**), it informs the user that an account is already registered to that CNIC.
5. **Navigation Options:**
 - It provides options to the user:
 - **1** to go back and try entering information again (**goto start2;**),
 - **2** to go back to the main menu (**goto start;**),
 - **3** to exit the program.
6. **Handling Navigation:**
 - Depending on the user's choice, the program either goes back to entering information (**start2**), back to the main menu (**start**), or exits the program.
7. **Registering a New Account:**
 - If no existing account is found for the given CNIC, the program searches for an empty slot in the array (**cnic**) to register a new account.
 - It generates a random account number (**accnum**) while ensuring it is unique among existing accounts.

- The program then prompts the user to create a 4-digit PIN. If a valid PIN is provided, the account details (name, CNIC, account number, and PIN) are stored in the respective arrays (**name**, **cnic**, **account_number**, and **pin**).

8. Repetition of Process:

- The use of **goto** allows for a simplified way of repeating the process if the user provides invalid input (e.g., a CNIC already associated with an account or an invalid PIN).

This case allows the user to open a new account by providing their name and a 3-digit CNIC. It ensures that the CNIC is unique and, if not, provides options for the user to navigate. If the user provides valid information, it generates an account number and prompts them to create a PIN for the new account. The use of **goto** allows for easy repetition of the process in case of invalid input.

Case 3: Close an Account

case 3:

start3:

```

    userindex = -1;
    cout << "Enter Your Account Number: ";
    cin >> enter_accnum;

    for (int i = 0; i < max_users; i++) {
        if (enter_accnum == account_number[i]) {
            userindex = i;
            break;
        }
    }

    if (userindex != -1) {
        char sure;
        confirm:
            cout << "Are you sure you want to delete your account? (y/n): ";
            cin >> sure;

        if (sure == 'y' || sure == 'Y') {
            name[userindex] = "";
            cnic[userindex] = 0;
            account_number[userindex] = 0;
            pin[userindex] = 0;
            cout << "Account Deleted!" << endl << endl;
            goto start;
        } else if (sure == 'n' || sure == 'N') {
            cout << "Account Not Deleted!" << endl << endl;
            goto start;
        } else {

```

```

        cout << "Invalid Option!" << endl;
        goto confirm;
    }
} else {
    cout << "No Account Found!" << endl << endl;

    nav3:
        cout << "\n\n1 Back \n2 Main Menu \n3 Exit \nSelect Option: ";
        cin >> option;

    switch (option) {
        case 1:
            cout << endl;
            goto start3;
            break;
        case 2:
            cout << endl;
            goto start;
            break;
        case 3:
            cout << "Exiting System!";
            return 0;
            break;
        default:
            cout << "Invalid Selection!" << endl;;
            goto nav3;
            break;
    }
}
break;

```

Explanation:

1. **Initialization:**
 - **userindex** is initialized to -1 to indicate that the user hasn't been found initially.
2. **User Input:**
 - The program prompts the user to enter their account number.
3. **Search for Account:**
 - It then checks if there is an account with the provided account number.
4. **Account Found:**
 - If an account is found (**userindex != -1**), it asks the user for confirmation to delete the account.
5. **User Confirmation:**
 - It prompts the user with a 'yes' or 'no' question regarding the deletion of their account.

- If the user confirms with 'y' or 'Y', it deletes the account by resetting the corresponding details (name, CNIC, account number, and PIN) to default values.
6. **Account Not Deleted:**
 - If the user chooses not to delete the account (**sure == 'n' || sure == 'N'**), it informs the user that the account is not deleted.
 7. **Invalid Option Handling:**
 - If the user provides an invalid option, it informs the user and goes back to the confirmation step.
 8. **No Account Found:**
 - If no account is found for the given account number, it informs the user.
 9. **Navigation Options:**
 - It provides options to the user:
 - **1** to go back and try entering the account number again (**goto start3;**),
 - **2** to go back to the main menu (**goto start;**),
 - **3** to exit the program.
 10. **Handling Navigation:**
 - Depending on the user's choice, the program either goes back to entering the account number (**start3**), back to the main menu (**start**), or exits the program.

This case allows the user to close their account by providing the account number. It ensures user confirmation before deleting the account. The use of **goto** allows for easy repetition in case of invalid input or if the user decides not to delete the account.

Case 4: Show Balance

case 4:

```
start4:
    userindex = -1;
    cout << "Enter Account Number: ";
    cin >> enter_accnum;

    for (int i = 0; i < max_users; i++) {
        if (enter_accnum == account_number[i]) {
            userindex = i;
            break;
        }
    }

    if (userindex != -1) {
        retry1:
            cout << "Enter Account Pin: ";
            cin >> enter_pin;

            if (enter_pin == pin[userindex]) {
                sub_start4:
                    cout << "1 Current \n2 Default \n3 Savings \nSelect Account: ";
```

```

        cin >> access;

switch (access) {
    case 1:
        cout << "Current Account Balance: " << current[userindex];
        break;
    case 2:
        cout << "Default Account Balance: " << defaults[userindex];
        break;
    case 3:
        cout << "Savings Account Balance: " << savings[userindex];
        break;
    default:
        cout << "Invalid Selection!";
}

nav4_1:
    cout << "\n\n1 Back \n2 Main Menu \n3 Exit \nSelect Option: ";
    cin >> option;

switch (option) {
    case 1:
        cout << endl;
        goto sub_start4;
        break;
    case 2:
        cout << endl;
        goto start;
        break;
    case 3:
        cout << "Exiting System!";
        return 0;
        break;
    default:
        cout << "Invalid Selection!" << endl;;
        goto nav4_1;
        break;
}
} else {
    cout << "Invalid Pin!" << endl << endl;
    goto retry1;
}
} else {
    cout << "No Account Found!" << endl << endl;

nav4_2:

```

```

        cout << "1 Back \n2 Main Menu \n3 Exit \nSelect Option: ";
        cin >> option;

        switch (option) {
            case 1:
                cout << endl;
                goto start4;
                break;
            case 2:
                cout << endl;
                goto start;
                break;
            case 3:
                cout << "Exiting System!";
                return 0;
                break;
            default:
                cout << "Invalid Selection!" << endl;;
                goto nav4_2;
                break;
        }
    }
    break;

```

Explanation:

1. **Initialization:**
 - **userid** is initialized to -1 to indicate that the user hasn't been found initially.
2. **User Input:**
 - The program prompts the user to enter their account number.
3. **Search for Account:**
 - It then checks if there is an account with the provided account number.
4. **Account Found:**
 - If an account is found (**userid != -1**), it prompts the user to enter their account PIN for verification.
5. **PIN Verification:**
 - If the entered PIN matches the stored PIN, it displays the balance for different account types (current, default, savings).
6. **Displaying Balance:**
 - It prompts the user to select the type of account (current, default, or savings) and displays the corresponding account balance.
7. **Navigation Options:**
 - It provides options to the user:
 - **1** to go back and select another account type (**goto sub_start4;**),
 - **2** to go back to the main menu (**goto start;**),

- **3** to exit the program.
8. **Handling Navigation:**
 - Depending on the user's choice, the program either goes back to selecting the account type (**sub_start4**), back to the main menu (**start**), or exits the program.
 9. **Invalid PIN Handling:**
 - If the entered PIN is invalid, it informs the user and allows them to retry (**goto retry1;**).
 10. **No Account Found:**
 - If no account is found for the given account number, it informs the user.
 11. **Navigation Options (No Account Found):**
 - It provides options to the user:
 - **1** to go back and try entering the account number again (**goto start4;**),
 - **2** to go back to the main menu (**goto start;**),
 - **3** to exit the program.
 12. **Handling Navigation (No Account Found):**
 - Depending on the user's choice, the program either goes back to entering the account number (**start4**), back to the main menu (**start**), or exits the program.

This case allows the user to check the balance of their account by providing the account number and PIN. It provides options for navigating through different account types and allows the user to go back or exit the program. The use of **goto** allows for easy repetition in case of invalid input or if the user decides to try another option.

Case 5: Deposit Amount

case 5:

```
start5:
    userindex = -1;
    cout << "Enter Account Number: ";
    cin >> enter_accnum;

    for (int i = 0; i < max_users; i++) {
        if (enter_accnum == account_number[i]) {
            userindex = i;
            break;
        }
    }

    if (userindex != -1) {
        retry2:
            cout << "Enter Account Pin: ";
            cin >> enter_pin;

            if (enter_pin == pin[userindex]) {
                sub_start5:
```

```

        cout << "1 Current \n2 Default \n3 Savings \nSelect Account In
Which You Want To Deposit Amount: ";
        cin >> access;
        cout << "Enter Amount That You Want To Deposit: ";
        cin >> deposit;

```

```

switch (access) {
    case 1:
        current[userindex] += deposit;
        cout << "Current Account Balance: " << current[userindex];
        break;
    case 2:
        defaults[userindex] += deposit;
        cout << "Default Account Balance: " << defaults[userindex];
        break;
    case 3:
        savings[userindex] += deposit;
        cout << "Savings Account Balance: " << savings[userindex];
        break;
    default:
        cout << "Invalid Selection!";
}

```

```

nav5_1:
    cout << "\n\n1 Back \n2 Main Menu \n3 Exit \nSelect Option: ";
    cin >> option;

```

```

switch (option) {
    case 1:
        cout << endl;
        goto sub_start5;
        break;
    case 2:
        cout << endl;
        goto start;
        break;
    case 3:
        cout << "Exiting System!";
        return 0;
        break;
    default:
        cout << "Invalid Selection!" << endl;
        goto nav5_1;
        break;
}

```

```

} else {

```

```

        cout << "Invalid Pin!" << endl << endl;
        goto retry2;
    }
} else {
    cout << "No Account Found!" << endl << endl;

    nav5_2:
        cout << "1 Back \n2 Main Menu \n3 Exit \nSelect Option: ";
        cin >> option;

        switch (option) {
            case 1:
                cout << endl;
                goto start5;
                break;
            case 2:
                cout << endl;
                goto start;
                break;
            case 3:
                cout << "Exiting System!";
                return 0;
                break;
            default:
                cout << "Invalid Selection!" << endl;
                goto nav5_2;
                break;
        }
    }
    break;
}

```

Explanation:

1. **Initialization:**
 - **userid** is initialized to -1 to indicate that the user hasn't been found initially.
2. **User Input:**
 - The program prompts the user to enter their account number.
3. **Search for Account:**
 - It then checks if there is an account with the provided account number.
4. **Account Found:**
 - If an account is found (**userid != -1**), it prompts the user to enter their account PIN for verification.
5. **PIN Verification:**
 - If the entered PIN matches the stored PIN, it prompts the user to select the account type (current, default, or savings) in which they want to deposit an amount.

6. **Deposit Amount:**

- It prompts the user to enter the amount they want to deposit.
- Depending on the selected account type, it updates the corresponding balance (**current**, **defaults**, or **savings**) with the deposited amount.

7. **Displaying Updated Balance:**

- It displays the updated balance for the selected account type.

8. **Navigation Options:**

- It provides options to the user:
 - **1** to go back and deposit in another account type (**goto sub_start5;**),
 - **2** to go back to the main menu (**goto start;**),
 - **3** to exit the program.

9. **Handling Navigation:**

- Depending on the user's choice, the program either goes back to selecting the account type (**sub_start5**), back to the main menu (**start**), or exits the program.

10. **Invalid PIN Handling:**

- If the entered PIN is invalid, it informs the user and allows them to retry (**goto retry2;**).

11. **No Account Found:**

- If no account is found for the given account number, it informs the user.

12. **Navigation Options (No Account Found):**

- It provides options to the user:
 - **1** to go back and try entering the account number again (**goto start5;**),
 - **2** to go back to the main menu (**goto start;**),
 - **3** to exit the program.

13. **Handling Navigation (No Account Found):**

- Depending on the user's choice, the program either goes back to entering the account number (**start5**), back to the main menu (**start**), or exits the program.

This case allows the user to deposit an amount into their account by providing the account number and PIN. It prompts the user to select the account type and displays the updated balance. The use of **goto** allows for easy repetition in case of invalid input or if the user decides to try another option.

Case 6: Withdraw Amount

case 6:

```
start6:
    userindex = -1;
    cout << "Enter Account Number: ";
    cin >> enter_accnum;

    for (int i = 0; i < max_users; i++) {
        if (enter_accnum == account_number[i]) {
            userindex = i;
            break;
        }
    }
```

```

    }
}

if (userindex != -1) {
    retry3:
        cout << "Enter Account Pin: ";
        cin >> enter_pin;

        if (enter_pin == pin[userindex]) {
            sub_start6:
                cout << "1 Current \n2 Default \n3 Savings \nSelect Account
From Which You Want To Withdraw Amount: ";
                cin >> access;
                cout << "Enter Amount That You Want To Withdraw: ";
                cin >> withdraw;

                switch (access) {
                    case 1:
                        if (withdraw > current[userindex]) {
                            cout << "Insufficient Balance!";
                        } else {
                            current[userindex] -= withdraw;
                            cout << "Current Account Balance: " <<
current[userindex];
                        }
                        break;
                    case 2:
                        if (withdraw > defaults[userindex]) {
                            cout << "Insufficient Balance!";
                        } else {
                            defaults[userindex] -= withdraw;
                            cout << "Default Account Balance: " <<
defaults[userindex];
                        }
                        break;
                    case 3:
                        if (withdraw > savings[userindex]) {
                            cout << "Insufficient Balance!";
                        } else {
                            savings[userindex] -= withdraw;
                            cout << "Savings Account Balance: " <<
savings[userindex];
                        }
                        break;
                    default:
                        cout << "Invalid Selection!";
                }
            }
        }
    }
}

```



```

    }

    nav6_1:
        cout << "\n\n1 Back \n2 Main Menu \n3 Exit \nSelect Option: ";
        cin >> option;

        switch (option) {
            case 1:
                cout << endl;
                goto sub_start6;
                break;
            case 2:
                cout << endl;
                goto start;
                break;
            case 3:
                cout << "Exiting System!";
                return 0;
                break;
            default:
                cout << "Invalid Selection!" << endl;
                goto nav6_1;
                break;
        }
    } else {
        cout << "Invalid Pin!" << endl << endl;
        goto retry3;
    }
} else {
    cout << "No Account Found!" << endl << endl;

    nav6_2:
        cout << "1 Back \n2 Main Menu \n3 Exit \nSelect Option: ";
        cin >> option;

        switch (option) {
            case 1:
                cout << endl;
                goto start6;
                break;
            case 2:
                cout << endl;
                goto start;
                break;
            case 3:
                cout << "Exiting System!";

```

```

        return 0;
        break;
    default:
        cout << "Invalid Selection!" << endl;
        goto nav6_2;
        break;
    }
}
break;

```

Explanation:

1. **Initialization:**
 - **userindex** is initialized to -1 to indicate that the user hasn't been found initially.
2. **User Input:**
 - The program prompts the user to enter their account number.
3. **Search for Account:**
 - It then checks if there is an account with the provided account number.
4. **Account Found:**
 - If an account is found (**userindex != -1**), it prompts the user to enter their account PIN for verification.
5. **PIN Verification:**
 - If the entered PIN matches the stored PIN, it prompts the user to select the account type (current, default, or savings) from which they want to withdraw an amount.
6. **Withdraw Amount:**
 - It prompts the user to enter the amount they want to withdraw.
 - Depending on the selected account type, it checks if the withdrawal amount is within the available balance.
 - If sufficient balance is available, it deducts the withdrawal amount from the corresponding balance (**current**, **defaults**, or **savings**).
7. **Displaying Updated Balance or Insufficient Balance Message:**
 - It either displays the updated balance or informs the user that there is insufficient balance for the withdrawal.
8. **Navigation Options:**
 - It provides options to the user:
 - **1** to go back and withdraw from another account type (**goto sub_start6;**),
 - **2** to go back to the main menu (**goto start;**),
 - **3** to exit the program.
9. **Handling Navigation:**
 - Depending on the user's choice, the program either goes back to selecting the account type (**sub_start6**), back to the main menu (**start**), or exits the program.
10. **Invalid PIN Handling:**

- If the entered PIN is invalid, it informs the user and allows them to retry (**goto retry3;**).

11. No Account Found:

- If no account is found for the given account number, it informs the user.

12. Navigation Options (No Account Found):

- It provides options to the user:
 - **1** to go back and try entering the account number again (**goto start6;**),
 - **2** to go back to the main menu (**goto start;**),
 - **3** to exit the program.

13. Handling Navigation (No Account Found):

- Depending on the user's choice, the program either goes back to entering the account number (**start6**), back to the main menu (**start**), or exits the program.

This case allows the user to withdraw an amount from their account by providing the account number and PIN. It prompts the user to select the account type and displays the updated balance or an insufficient balance message. The use of **goto** allows for easy repetition in case of invalid input or if the user decides to try another option.

Case 7: Exit

case 7:

```
cout<<"Exiting System!";
break;
```

Explanation:

This case is the exit option in the banking system menu. When the user selects option 7, it prints the message "Exiting System!" and then breaks out of the switch-case statement, effectively ending the program.

Default Case

default:

```
cout << "Invalid Selection!" << endl << endl;
goto start;
```

Explanation:

The **default** case in a switch statement is executed when none of the specified cases match the value of the expression being evaluated. In this banking system program, the **switch** statement is used to handle user input for various banking operations. The **default** case is a catch-all option for handling invalid or unexpected user input.

Conclusion

The provided C++ code represents a simple banking system that allows users to perform various operations such as checking account details, opening new accounts, closing accounts, checking balances, depositing amounts, and withdrawing funds. The program utilizes a switch-case structure to handle different user choices. While the code achieves its intended functionality, there are several points to consider:

1. **Goto Statements:**

- The code extensively uses **goto** statements for navigation. While **goto** can be a quick solution, it is generally discouraged in modern programming due to its impact on code readability and maintainability. It can lead to spaghetti code and make debugging more challenging.

2. **Code Organization:**

- The code lacks modularity and separation of concerns. Functions could be introduced to handle specific tasks, making the code more structured and easier to understand.

3. **Input Validation:**

- The code does not include robust input validation. For example, it assumes that users will always enter valid data. Implementing input validation can enhance the program's robustness and prevent unexpected behavior.

4. **Error Handling:**

- The code lacks comprehensive error handling. It would be beneficial to include mechanisms to handle unexpected scenarios, such as file I/O errors or memory allocation failures.

5. **Security Concerns:**

- Storing sensitive information like PINs in plain arrays is not recommended for a real-world banking system. In practice, secure practices such as encryption and secure storage should be employed.

6. **User Experience:**

- The user interface could be improved to provide clearer prompts and feedback. Additionally, incorporating a loop for the main menu would allow users to perform multiple operations without restarting the program.

7. **Comments and Documentation:**

- The code lacks comments and documentation, making it difficult for other developers (or even the original author) to understand the code's logic and purpose.

In conclusion, while the code achieves the basic functionality of a simple banking system.